W4111 Web Application Notebook

This notebook calls the `/customers`, `/orders`, and `/orderdetails` routes from `app/main.py` over HTTP using the `requests` package.

Prerequisite: Run the API from the repository root, for example `uvicorn app.main:app --reload --port 8000`. The default base URL is `http://127.0.0.1:8000` (as used below); override with the environment variable `API_BASE_URL` if you use another host or port.

Note: Cells that `POST`, `PUT`, or `DELETE` modify your local `classicmodels` database records. Run create/update/delete cells carefully and preferably only on temporary rows (for example, customer or order `99999`).

In [1]:
```
import requests
from pprint import pprint
import json
import os

BASE_URL = os.getenv("API_BASE_URL", "http://127.0.0.1:8000")

def show_response(resp, max_items=2):
    print("Status:", resp.status_code)
    try:
        data = resp.json()
        if isinstance(data, dict) and "items" in data and isinstance(data["i
            print("Item count:", len(data["items"]))
            pprint(data["items"][:max_items])
        else:
            pprint(data)
    except Exception:
        print(resp.text)
```

In [2]:
```
requests.delete(f"{BASE_URL}/orders/99999/orderdetails/S18_1749")
requests.delete(f"{BASE_URL}/orders/99999")
requests.delete(f"{BASE_URL}/customers/99999")
print("Cleanup done !")
```

```
Cleanup done !
```

In [3]:
```
resp = requests.get("http://127.0.0.1:8000/health")
show_response(resp)
```

```
Status: 200
{'status': 'ok'}
```

In [4]:
```
def show_response(resp, max_items=2):
    print("Status:", resp.status_code)
    try:
        data = resp.json()
```

```
        if isinstance(data, dict) and "items" in data and isinstance(data["i
            print("Item count:", len(data["items"]))
            pprint(data["items"][:max_items])
        else:
            pprint(data)
    except Exception:
        print(resp.text)
```

In [5]:
```
resp = requests.get(f"{BASE_URL}/health")
show_response(resp)
```

```
Status: 200
{'status': 'ok'}
```

GET /customers

Shows all customers, or filters them based on any query parameters you include.

In [6]:
```
resp = requests.get(f"{BASE_URL}/customers")
show_response(resp)
```

```
Status: 200
Item count: 122
[{'addressLine1': '725 Downing Street',
  'addressLine2': None,
  'city': 'London',
  'contactFirstName': 'Diana ',
  'contactLastName': 'Wimbledon',
  'country': 'United Kingdom',
  'creditLimit': 21000.0,
  'customerName': 'Something somewhere',
  'customerNumber': 103,
  'phone': '+44-111-222-3333',
  'postalCode': 'SW1A 1AA',
  'salesRepEmployeeNumber': 1370,
  'state': None},
 {'addressLine1': '8489 Strong St.',
  'addressLine2': None,
  'city': 'Las Vegas',
  'contactFirstName': 'Jean',
  'contactLastName': 'King',
  'country': 'USA',
  'creditLimit': 71800.0,
  'customerName': 'Signal Gift Stores',
  'customerNumber': 112,
  'phone': '7025551838',
  'postalCode': '83030',
  'salesRepEmployeeNumber': 1166,
  'state': 'NV'}]
```

In [7]:
```
resp = requests.get(f"{BASE_URL}/customers", params={"country": "France"})
show_response(resp)
```

```
Status: 200
Item count: 11
[{'addressLine1': '67, rue des Cinquante Otages',
  'addressLine2': None,
  'city': 'Nantes',
  'contactFirstName': 'Janine ',
  'contactLastName': 'Labrune',
  'country': 'France',
  'creditLimit': 118200.0,
  'customerName': 'La Rochelle Gifts',
  'customerNumber': 119,
  'phone': '40.67.8555',
  'postalCode': '44000',
  'salesRepEmployeeNumber': 1370,
  'state': None},
 {'addressLine1': '2, rue du Commerce',
  'addressLine2': None,
  'city': 'Lyon',
  'contactFirstName': 'Mary ',
  'contactLastName': 'Saveley',
  'country': 'France',
  'creditLimit': 123900.0,
  'customerName': 'Saveley & Henriot, Co.',
  'customerNumber': 146,
  'phone': '78.32.5555',
  'postalCode': '69004',
  'salesRepEmployeeNumber': 1337,
  'state': None}]
```

`GET /customers/{customerNumber}`

Returns **`404`** if the customer does not exist.

In [8]:
```
resp = requests.get(f"{BASE_URL}/customers/103")
show_response(resp)
```

```
Status: 200
{'addressLine1': '725 Downing Street',
 'addressLine2': None,
 'city': 'London',
 'contactFirstName': 'Diana ',
 'contactLastName': 'Wimbledon',
 'country': 'United Kingdom',
 'creditLimit': 21000.0,
 'customerName': 'Something somewhere',
 'customerNumber': 103,
 'phone': '+44-111-222-3333',
 'postalCode': 'SW1A 1AA',
 'salesRepEmployeeNumber': 1370,
 'state': None}
```

`PUT /customers/{customerNumber}`

Updates a customer by customerNumber. Returns the update result from the API.

In [9]:
```
customer_103 = {
    "customerNumber": 103,
    "customerName": "Something somewhere",
    "contactLastName": "Wimbledon",
    "contactFirstName": "Diana ",
    "phone": "+44-111-222-3333",
    "addressLine1": "725 Downing Street",
    "addressLine2": None,
    "city": "London",
    "state": None,
    "postalCode": "SW1A 1AA",
    "country": "United Kingdom",
    "salesRepEmployeeNumber": 1370,
    "creditLimit": 21000
}

resp = requests.put(f"{BASE_URL}/customers/103", json=customer_103)
show_response(resp)
```

```
Status: 200
{'updated': 0}
```

In [10]:
```
resp = requests.get(f"{BASE_URL}/customers/103")
show_response(resp)
```

```
Status: 200
{'addressLine1': '725 Downing Street',
 'addressLine2': None,
 'city': 'London',
 'contactFirstName': 'Diana ',
 'contactLastName': 'Wimbledon',
 'country': 'United Kingdom',
 'creditLimit': 21000.0,
 'customerName': 'Something somewhere',
 'customerNumber': 103,
 'phone': '+44-111-222-3333',
 'postalCode': 'SW1A 1AA',
 'salesRepEmployeeNumber': 1370,
 'state': None}
```

`POST /customers`

Creates a customer. Use a temporary customer number for testing so that the notebook is able to rerun easily.

In [11]:
```
temp_customer = {
    "customerNumber": 99999,
    "customerName": "Test Customer",
    "contactLastName": "Fajemisin",
    "contactFirstName": "Anjola",
    "phone": "+1-555-123-4567",
    "addressLine1": "W4111 ILoveDatabases St",
```

```
    "addressLine2": None,
    "city": "New York",
    "state": "NY",
    "postalCode": "10027",
    "country": "USA",
    "salesRepEmployeeNumber": 1370,
    "creditLimit": 50000
}

resp = requests.post(f"{BASE_URL}/customers", json=temp_customer)
show_response(resp)
```

```
Status: 201
'0'
```

In [12]:

```
resp = requests.get(f"{BASE_URL}/customers/99999")
show_response(resp)
```

```
Status: 200
{'addressLine1': 'W4111 ILoveDatabases St',
 'addressLine2': None,
 'city': 'New York',
 'contactFirstName': 'Anjola',
 'contactLastName': 'Fajemisin',
 'country': 'USA',
 'creditLimit': 50000.0,
 'customerName': 'Test Customer',
 'customerNumber': 99999,
 'phone': '+1-555-123-4567',
 'postalCode': '10027',
 'salesRepEmployeeNumber': 1370,
 'state': 'NY'}
```

`GET /orders`

List/search with optional query parameters acting as an equality template over order fields.

In [13]:

```
resp = requests.get(f"{BASE_URL}/orders")
show_response(resp)
```

```
Status: 200
Item count: 326
[{'comments': None,
  'customerNumber': 363,
  'orderDate': '2003-01-06',
  'orderNumber': 10100,
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped'},
 {'comments': 'Check on availability.',
  'customerNumber': 128,
  'orderDate': '2003-01-09',
  'orderNumber': 10101,
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped'}]
```

In [14]:
```
resp = requests.get(f"{BASE_URL}/orders", params={"customerNumber": 103})
show_response(resp)
```

```
Status: 200
Item count: 3
[{'comments': None,
  'customerNumber': 103,
  'orderDate': '2003-05-20',
  'orderNumber': 10123,
  'requiredDate': '2003-05-29',
  'shippedDate': '2003-05-22',
  'status': 'Shipped'},
 {'comments': None,
  'customerNumber': 103,
  'orderDate': '2004-09-27',
  'orderNumber': 10298,
  'requiredDate': '2004-10-05',
  'shippedDate': '2004-10-01',
  'status': 'Shipped'}]
```

In [15]:
```
resp = requests.get(f"{BASE_URL}/orders", params={"status": "Shipped"})
show_response(resp)
```

```
Status: 200
Item count: 303
[{'comments': None,
  'customerNumber': 363,
  'orderDate': '2003-01-06',
  'orderNumber': 10100,
  'requiredDate': '2003-01-13',
  'shippedDate': '2003-01-10',
  'status': 'Shipped'},
 {'comments': 'Check on availability.',
  'customerNumber': 128,
  'orderDate': '2003-01-09',
  'orderNumber': 10101,
  'requiredDate': '2003-01-18',
  'shippedDate': '2003-01-11',
  'status': 'Shipped'}]
```

GET /orders/{orderNumber}

Returns **404** if the order does not exist.

In [16]:
```
resp = requests.get(f"{BASE_URL}/orders/10100")
show_response(resp)
```

```
Status: 200
{'comments': None,
 'customerNumber': 363,
 'orderDate': '2003-01-06',
 'orderNumber': 10100,
 'requiredDate': '2003-01-13',
 'shippedDate': '2003-01-10',
 'status': 'Shipped'}
```

GET /orderdetails

Shows all order details, or filters them based on any query parameters you include.

In [17]:
```
resp = requests.get(f"{BASE_URL}/orderdetails", params={"orderNumber": 10100
show_response(resp, max_items=10)
```

```
Status: 200
Item count: 4
[{'orderLineNumber': 3,
  'orderNumber': 10100,
  'priceEach': 136.0,
  'productCode': 'S18_1749',
  'quantityOrdered': 30},
 {'orderLineNumber': 2,
  'orderNumber': 10100,
  'priceEach': 55.09,
  'productCode': 'S18_2248',
  'quantityOrdered': 50},
 {'orderLineNumber': 4,
  'orderNumber': 10100,
  'priceEach': 75.46,
  'productCode': 'S18_4409',
  'quantityOrdered': 22},
 {'orderLineNumber': 1,
  'orderNumber': 10100,
  'priceEach': 35.29,
  'productCode': 'S24_3969',
  'quantityOrdered': 49}]
```

In [18]:
```
resp = requests.get(f"{BASE_URL}/orderdetails")
show_response(resp)
```

```
Status: 200
Item count: 2996
[{'orderLineNumber': 3,
  'orderNumber': 10100,
  'priceEach': 136.0,
  'productCode': 'S18_1749',
  'quantityOrdered': 30},
 {'orderLineNumber': 2,
  'orderNumber': 10100,
  'priceEach': 55.09,
  'productCode': 'S18_2248',
  'quantityOrdered': 50}]
```

`GET /customers/{customerNumber} (missing)`

Returns `404` when the customer does not exist.

In [19]:
```
resp = requests.get(f"{BASE_URL}/customers/999999")
print("Status:", resp.status_code)
print(resp.text)
```

```
Status: 404
{"detail":"No customer with customerNumber 999999"}
```

`GET /orders/{orderNumber}` (missing)

Returns **`404`** and an error body when the order does not exist.

In [20]:
```
resp = requests.get(f"{BASE_URL}/orders/999999", timeout=30)
print("Status:", resp.status_code)
print(resp.json())
```

```
Status: 404
{'detail': 'No order with orderNumber 999999'}
```

`DELETE /customers/{customerNumber}`

Deletes the temporary test customer and returns whether a row was removed.

In [21]:
```
resp = requests.delete(f"{BASE_URL}/customers/99999")
show_response(resp)
```

```
Status: 200
{'deleted': 1}
```

In [22]:
```
resp = requests.get(f"{BASE_URL}/customers/99999")
print("Status:", resp.status_code)
print(resp.text)
```

```
Status: 404
{"detail":"No customer with customerNumber 99999"}
```

Orders: create, update, delete

The next few cells test the full CRUD flow for orders:

- `POST /orders`
- `GET /orders/{orderNumber}`
- `PUT /orders/{orderNumber}`
- `DELETE /orders/{orderNumber}`

In [23]:
```
temp_order = {
    "orderNumber": 99999,
    "orderDate": "2026-05-05",
    "requiredDate": "2026-05-12",
    "shippedDate": "2026-05-06",
    "status": "Shipped",
    "comments": "Temporary test order !",
    "customerNumber": 103
}
resp = requests.post(f"{BASE_URL}/orders", json=temp_order)
show_response(resp)
```

```
Status: 201
'0'
```

In [24]:
```
resp = requests.get(f"{BASE_URL}/orders/99999")
show_response(resp)
```

```
Status: 200
{'comments': 'Temporary test order !',
 'customerNumber': 103,
 'orderDate': '2026-05-05',
 'orderNumber': 99999,
 'requiredDate': '2026-05-12',
 'shippedDate': '2026-05-06',
 'status': 'Shipped'}
```

In [25]:
```
updated_order = dict(temp_order)
updated_order["status"] = "Cancelled"
updated_order["comments"] = "Updated"

resp = requests.put(f"{BASE_URL}/orders/99999", json=updated_order)
show_response(resp)
```

```
Status: 200
{'updated': 1}
```

In [26]:
```
resp = requests.get(f"{BASE_URL}/orders/99999")
show_response(resp)
```

```
Status: 200
{'comments': 'Updated',
 'customerNumber': 103,
 'orderDate': '2026-05-05',
 'orderNumber': 99999,
 'requiredDate': '2026-05-12',
 'shippedDate': '2026-05-06',
 'status': 'Cancelled'}
```

OrderDetails: create, update, delete

The `orderdetails` table uses a composite primary key (`orderNumber` , `productCode`).

These cells test:

- `POST /orderdetails`
- `GET /orders/{orderNumber}/orderdetails/{productCode}`
- `PUT /orders/{orderNumber}/orderdetails/{productCode}`
- `DELETE /orders/{orderNumber}/orderdetails/{productCode}`

using the temporary order `99999` .

In [27]:
```
temp_order_detail = {
    "orderNumber": 99999,
    "productCode": "S18_1749",
    "quantityOrdered": 5,
    "priceEach": 136.00,
    "orderLineNumber": 1
}

resp = requests.post(f"{BASE_URL}/orderdetails", json=temp_order_detail)
show_response(resp)
```

```
Status: 201
'99999|S18_1749'
```

In [28]:
```
resp = requests.get(f"{BASE_URL}/orders/99999/orderdetails/S18_1749")
show_response(resp)
```

```
Status: 200
{'orderLineNumber': 1,
 'orderNumber': 99999,
 'priceEach': 136.0,
 'productCode': 'S18_1749',
 'quantityOrdered': 5}
```

In [29]:
```
updated_order_detail = dict(temp_order_detail)
updated_order_detail["quantityOrdered"] = 8
updated_order_detail["priceEach"] = 140.00

resp = requests.put(
    f"{BASE_URL}/orders/99999/orderdetails/S18_1749",
```

```
    json=updated_order_detail
)
show_response(resp)
```

```
Status: 200
{'updated': 1}
```

In [30]:
```
resp = requests.get(f"{BASE_URL}/orders/99999/orderdetails/S18_1749")
show_response(resp)
```

```
Status: 200
{'orderLineNumber': 1,
 'orderNumber': 99999,
 'priceEach': 140.0,
 'productCode': 'S18_1749',
 'quantityOrdered': 8}
```

In [31]:
```
resp = requests.delete(f"{BASE_URL}/orders/99999/orderdetails/S18_1749")
show_response(resp)
```

```
Status: 200
{'deleted': 1}
```

In [32]:
```
resp = requests.get(f"{BASE_URL}/orders/99999/orderdetails/S18_1749")
print("Status:", resp.status_code)
print(resp.text)
```

```
Status: 404
{"detail":"No orderdetail with orderNumber='99999', productCode='S18_1749'"}
```

In [33]:
```
resp = requests.delete(f"{BASE_URL}/orders/99999")
show_response(resp)
```

```
Status: 200
{'deleted': 1}
```

In [34]:
```
resp = requests.get(f"{BASE_URL}/orders/99999")
print("Status:", resp.status_code)
print(resp.text)
```

```
Status: 404
{"detail":"No order with orderNumber 99999"}
```

In [35]:
```
resp = requests.delete(f"{BASE_URL}/orders/99999")
show_response(resp)
```

```
Status: 200
{'deleted': 0}
```

In [36]:
```
resp = requests.get(f"{BASE_URL}/orders/99999")
print("Status:", resp.status_code)
print(resp.text)
```

```
Status: 404
{"detail":"No order with orderNumber 99999"}
```

`GET /orders/{orderNumber}/orderdetails/{produ`

Gets one order-detail record using both `orderNumber` and `productCode`.

In [37]:
```
order_number = 10100
product_code = "S18_1749"

resp = requests.get(f"{BASE_URL}/orders/{order_number}/orderdetails/{product
show_response(resp)
```

```
Status: 200
{'orderLineNumber': 3,
 'orderNumber': 10100,
 'priceEach': 136.0,
 'productCode': 'S18_1749',
 'quantityOrdered': 30}
```

In [38]:
```
resp = requests.get(f"{BASE_URL}/orders/99999/orderdetails/S18_1749")
print("Status:", resp.status_code)
print(resp.text)
```

```
Status: 404
{"detail":"No orderdetail with orderNumber='99999', productCode='S18_1749'"}
```